A **distributed database** is a database in which data is stored across different physical locations.[1] It may be stored in multiple computers located in the same physical location (e.g. a data centre); or maybe dispersed over a network of interconnected computers. Unlike parallel systems, in which the processors are tightly coupled and constitute a single database system, a distributed database system consists of loosely coupled sites that share no physical components.

System administrators can distribute collections of data (e.g. in a database) across multiple physical locations. A distributed database can reside on organised network servers or decentralised independent computers on the Internet, on corporate intranets or extranets, or on other organisation networks. Because distributed databases store data across multiple computers, distributed databases may improve performance at end-user worksites by allowing transactions to be processed on many machines, instead of being limited to one.[2]

Two processes ensure that the distributed databases remain up-to-date and current: replication[3] and duplication.

1. Replication involves using specialized software that looks for changes in the distributive database. Once the changes have been identified, the replication process makes all the databases look the same. The replication process can be complex and time-consuming, depending on the size and number of the distributed databases. This process can also require much time and computer resources.
2. Duplication, on the other hand, has less complexity. It identifies one database as a master and then duplicates that database. The duplication process is normally done at a set time after hours. This is to ensure that each distributed location has the same data. In the duplication process, users may change only the master database. This ensures that local data will not be overwritten.

Both replication and duplication can keep the data current in all distributive locations.[2]

Besides distributed database replication and fragmentation, there are many other distributed database design technologies. For example, local autonomy, synchronous, and asynchronous distributed database technologies. The implementation of these technologies can and do depend on the needs of the business and the sensitivity/confidentiality of the data stored in the database and the price the business is willing to spend on ensuring data security, consistency and integrity.

When discussing access to distributed databases, Microsoft favors the term **distributed query**, which it defines in protocol-specific manner as "[a]ny SELECT, INSERT, UPDATE, or DELETE statement that references tables and rowsets from one or more external OLE DB data sources."[4] Oracle provides a more language-centric view in which distributed queries and distributed transactions form part of **distributed SQL**.[5]

Architecture

There are 3 main architecture types for distributed databases:

- Shared-memory: very rarely used[6]
- Shared-disk
- Shared-nothing

In the shared-memory and shared-disk architectures, the data is not partitioned, but it has to be in a shared-nothing architecture.

Shared-disk architecture is more common for cloud databases than for on-premise.[6]

Historically, shared-nothing was the first architecture to be implemented on the cloud, before the advent of shared cloud storage made shared-disk possible.

In practice, different layers of the database can have different architectures. It is now common to have a compute layer with a shared nothing architecture, and a storage layer with a shared disk architecture. This is for instance the case of Snowflake[7] and AWS Aurora.[8]

List of shared-nothing databases

- IBM Db2
- Greenplum
- MongoDB
- Netezza
- Teradata
- TiDB
- OceanBase
- Vertica

List of shared-disk databases

- AWS Aurora

- Neon
- Snowflake

See also

- Centralized database
- Data grid
- Distributed cache
- Distributed data store
- Distributed hash table
- Routing protocol
- Distributed SQL